

theoretical forensics

Q1

1) ال size of object في ال memory بيتم بحسب ال member variables

تتأثر و جاز Data Alignment / padding
فالتالي وجود ال functions داخل ال class ال هيكلية ال
ال size يتأثر ال object

Ex

class class { *المتغيرات والبيانات*

public :

float x, y, z;

$3 \times 4 = 12 \text{ bytes}$

void move() {}

void render() {}

void update() {}

};

struct struct {

float x, y, z;

$3 \times 4 \text{ bytes} = 12 \text{ bytes}$

};

int main()

{ const $\&$ size of (class) ;

12 bytes

const $\&$ size of (struct) ;

12 bytes

2

مکان ال function نام memory

مقرن ف Code / Text segment

3

specific object ال Compiler معرفت ال

Pointer

~~Print~~

Q₂

the memory map

1 int global_player_count = 0, → initialized data

2 void calculatePhysics() → code / text seg-

3 Enemy * Boss = new Enemy(); →
stack ال pointer ال
Heap ال object ال Enemy ال

4 the actual 50 MB texture data request via
the new keyword
new → Heap

5

int temp_score = → stack
inside while

Q3 The Cliffhanger (Predict Session 2)

Resource Acquisition as initialization
Destructors

Q4 Security & Memory Bug in

Memory leak

void process (int dataSize)

```
{ if (dataSize <= 0)
{ cout << "invalid" << endl;
return 0;
}
```

بناءً على القول

نفس المكان
منه ما يتأخر

```
int * data Buffer = new int [dataSize];
delete [] data Buffer;
}
```

Q5 Encapsulation hack in

Private ← المخصوصة درجة الكرامة

Get Temp pointer (المستعمل نوعه)
One Temperature ← مؤتمن سياتر

memory ← ذاكرة

Breakage of Encapsulation

class NuclearReactor {

private :

double core-temperature ;

public :

void setTemperature (double temp)

{ if (temp > 1000.0)

{ cout << "high!" ; }

else { core-temperature = temp ; }

}

double : getTemperature () const

{ return core-temperature ; }

}

Q6 Mini Project

In Heap Manager

```
#include <stdio.h>
```

```
int * allocatescores (int size)
```

```
{  
    // ترجع مؤشر إلى الذاكرة المخصصة للبيانات  
    return new int [size];  
}
```

```
void freeScores (int * ptr)
```

```
{  
    // تستخدم delete  
    delete [] ptr;  
}
```

```
int main()
```

```
{  
    int size = 100;
```

```
    int * myScores = allocatescores (size);
```

```
    freeScores (myScores);
```

```
    cout << "Memory leak";
```

```
    for (int i = 0; i < 1000; i++)
```

```
    {  
        int * leakedPtr = allocatescores (100);
```

```
        // freeScores (leakedPtr);  
    }
```

```
    return 0;  
}
```

Q7

```
#include <iostream>
```

```
class Player {
```

```
public:
```

```
    int health = 100;
```

```
};
```

by Value void applyDamageValue (Player & P)

```
{ P.health -= 50; }
```

الذي يقلل 50 من 100

// By Pointer

```
void applyDamagePointer (Player * P)
```

```
{ if (P != nullptr)
```

```
{ P->health -= 50; }
```

```
}
```

عن الاصل بالPointer

```
void applyDamageReference (Player & P)
```

```
{ P.health -= 50; }
```

عن الاصل بالReference

```
int main ()
```

```
{ Player myPlayer;
```

applyDamageValue (my Player);
cout << myPlayer.health & " " << endl; → 100

applyDamagePointer (&myPlayer);
cout << myPlayer.health << endl; → 50

applyDamagePointer &
Reference (myPlayer);
cout << myPlayer.health << endl; → 0